

```

1 #include <systemc.h>
2 SC_MODULE(PriorityInterruptController)
3 {
4     sc_event interrupt_event;
5     bool irq[3];
6     int priority[3];
7
8     SC_CTOR(PriorityInterruptController)
9     {
10         // Priority: 0 = highest, 2 = lowest
11         priority[0] = 0;
12         priority[1] = 1;
13         priority[2] = 2;
14
15         irq[0] = false;
16         irq[1] = false;
17         irq[2] = false;
18
19         SC_THREAD(interrupt_handler);
20     }
21
22     // Generate an interrupt
23     void generate_interrupt(int device)
24     {
25         irq[device] = true;
26
27         cout << sc_time_stamp()
28             << " Device " << device
29             << " generated interrupt" << endl;
30
31         interrupt_event.notify();
32     }
33
34     // Interrupt handler
35     void interrupt_handler()
36     {
37         while (true)
38         {
39             wait(interrupt_event);
40
41             int highest_priority = -1;
42
43             // Find highest-priority pending interrupt
44             for (int i = 0; i < 3; i++)
45             {
46                 if (irq[i])
47                 {
48                     if (highest_priority == -1 ||
49                         priority[i] < priority[highest_priority])
50                     {
51                         highest_priority = i;
52                     }
53                 }
54             }
55
56             if (highest_priority != -1)
57             {
58                 cout << sc_time_stamp()
59                     << " Servicing Device "
60                     << highest_priority
61                     << " (Priority "
62                     << priority[highest_priority]
63                     << ")" << endl;
64
65                 irq[highest_priority] = false;
66
67                 // Interrupt service time
68                 wait(2, SC_NS);
69
70                 cout << sc_time_stamp()

```

```

58         cout << sc_time_stamp()
59             << " Servicing Device "
60             << highest_priority
61             << " (Priority "
62             << priority[highest_priority]
63             << ")" << endl;
64
65         irq[highest_priority] = false;
66
67         // Interrupt service time
68         wait(2, SC_NS);
69
70         cout << sc_time_stamp()
71             << " Device "
72             << highest_priority
73             << " interrupt completed"
74             << endl;
75     }
76 }
77 }
78 };
79
80
81 // Testbench
82 SC_MODULE(Testbench)
83 {
84     PriorityInterruptController* controller;
85
86     SC_CTOR(Testbench)
87     {
88         SC_THREAD(generate_interrupts);
89     }
90
91     void generate_interrupts()
92     {
93         wait(5, SC_NS);
94
95         controller->generate_interrupt(2);
96
97         wait(1, SC_NS);
98
99         controller->generate_interrupt(1);
100
101         wait(1, SC_NS);
102
103         controller->generate_interrupt(0);
104
105         wait(10, SC_NS);
106
107         sc_stop();
108     }
109 };
110
111
112 int sc_main(int argc, char* argv[])
113 {
114     PriorityInterruptController controller(
115         "PriorityInterruptController"
116     );
117
118     Testbench testbench("Testbench");
119
120     testbench.controller = &controller;
121
122     sc_start();
123
124     return 0;
125 }

```